

Problem: Checkout Continuity

Context

Gametime fans often switch devices during a purchase. A fan may discover tickets on mobile, send the event to a friend, then complete checkout on desktop. Another fan may start checkout on web, get interrupted, and come back through the app minutes before the event starts.

This is risky because checkout is time-sensitive. Inventory can disappear, prices can change, payment authorization can fail, and a partially completed order cannot be treated like a normal saved cart.

The Problem

We want fans to resume a purchase across web and mobile without creating duplicate orders, holding stale inventory forever, or hiding important changes like a price increase.

The experience has to coordinate three concerns:

- The backend needs a source of truth for the checkout session and its expiration.
- Web needs a fast, stable checkout surface that can render enough context before JavaScript finishes.
- Mobile needs to open the same session through a deep link and show platform-appropriate recovery states.

Your Challenge

Build a small checkout-continuity prototype.

Your solution should demonstrate:

1. Creating a checkout session for a selected listing.
2. Resuming that session from a second surface, such as a simulated mobile view or deep-link

route.

3. Handling at least two state changes that matter to the fan, such as inventory expiration, price change, payment pending, or completion failure.

You do not need to integrate with real payment, auth, or inventory systems. Stub them behind clear interfaces.

Some questions worth thinking through:

- What state lives on the backend, and what can safely live on the client?
- How do web and mobile know whether a session is still valid?
- What should the fan see if the price changed while they were away?
- How do you prevent duplicate orders when two devices resume the same session?
- How would you instrument this flow to know whether continuity improves checkout conversion?

Constraints and Notes

- Any language or framework is acceptable.
- In-memory storage is fine.
- Include at least a small API surface for creating, resuming, and completing a checkout session.
- The UI can be minimal, but it should make the cross-surface state transitions visible.

What We're Looking For

- A working prototype that shows the checkout session moving across surfaces
- A clear state model with expiration and failure handling
- Sound API boundaries between client surfaces and backend state
- Web performance judgment around what appears before hydration
- Tests or scripted scenarios for the important state transitions

Time Expectation

We expect this to take roughly 2-3 hours using modern development tools. Prefer a focused end-to-end slice over a broad but shallow checkout clone.

Submission

Create a GitHub repo with your solution. Include a README that explains:

- What you built and how to run it
- The checkout session state model
- How web and mobile resume the same session
- How you handle stale inventory, price changes, or duplicate completion
- What tradeoffs you made
- What you'd do differently with more time